Glyph Editor for Android

Customization guide: type a normal key, see your own glyph

1. What you have

An Android app written in **Kotlin**, Google's official language for Android and the simplest choice for this job. It uses **no extra libraries**, so the project stays small and easy to read. Pressing `a` on the keyboard shows **your picture for "a"**; every key you do not list stays normal.

hello → 

Key idea. The screen is one ordinary text box (`EditText`) that keeps the *real letters*. After each change, a small "span" is attached to every character that has a picture and tells Android: "draw this picture instead". Because of that, the keyboard, selection, copy/paste and saving all work normally, and files stay plain text.

Honest status. I wrote this app carefully and reviewed it line by line, but I had no Android SDK here, so **I could not build or run it**. The file format it reads (`glyphs.txt`) is the same one the Windows editor reads, and that part is tested. If Android Studio shows a red error, send me the exact message and I will fix it.

2. The folders

Path	What it is
`app/src/main/java/com/glypheditor/Config.kt`	**Your settings**: glyph folder, colours, title, padding.
`app/src/main/assets/glyphs/`	The glyph PNGs and `glyphs.txt` that get **built into the app** (your 18 samples are inside).
`MainActivity.kt`	The screen: text box, menu, open/save, glyph loading.
`GlyphSpan.kt`	"Draw a picture instead of a letter."
`GlyphMap.kt`	Reads `glyphs.txt` .
`tools/glyph_tool.py`	Cleans new pictures (same tool as the Windows project).
`build.gradle.kts, settings.gradle.kts, app/build.gradle.kts`	Build setup. You normally never touch these.

3. Build the app

1. Install **Android Studio** (free, from developer.android.com/studio). It brings the Android SDK with it.
2. Unzip the project. In Android Studio choose **File → Open** and select the `GlyphEditorAndroid` folder. Let it finish "Gradle sync" (it downloads what it needs; internet required the first time).
3. Plug in your phone (USB debugging on) or start an emulator, then press the green **Run ▶** button.
4. For a file you can share: **Build → Build Bundle(s)/APK(s) → Build APK(s)**. The file appears at `app/build/outputs/apk/debug/app-debug.apk`.

4. Make it portable (one APK file)

An APK is already a compressed package, and your glyph pictures are packed **inside** it (`assets/glyphs`). So the APK *is* the portable version: send it by USB, Bluetooth or WhatsApp, and on the other phone tap it to install (Android asks you to allow "install unknown apps"). No loose files are needed.

5. Tell it where your glyph folder is

The app looks in this order and uses the first place that contains `glyphs.txt`:

#	Place	Notes
1	`Config.GLYPH_FOLDER` (your own path)	The spot reserved for you in `Config.kt`. Outside the app's own area it needs the "All files access" permission on Android 11+.
2	The app's own folder `Android/data/com.glypheditor/files/glyphs`	Create it once (run the app one time, then connect the phone to a computer by USB and copy your PNGs and `glyphs.txt` in). **No rebuild needed.** Android 11+ hides `Android/data` from some file-manager apps, but USB and Android Studio's Device Explorer can reach it.
3	Built into the app (`assets/glyphs`)	The default. Change the files, then rebuild.

```
object Config {{
    // >>> 1. PUT THE FOLDER THAT HOLDS YOUR GLYPH PNGs + glyphs.txt HERE <<<
    const val GLYPH_FOLDER =
"/storage/emulated/0/Android/data/com.glypheditor/files/glyphs"
    ...
```

Help → About (menu "About / glyph folder") shows which of the three places was used and how many pictures loaded; the demo should say 18.

6. The list: `glyphs.txt` (same file as Windows)

 **a** = g01.png	 **b** = g02.png	 **c** = g03.png	 **d** = g04.png	 **e** = g05.png	 **f** = g06.png
 **g** = g07.png	 **h** = g08.png	 **i** = g09.png	 **j** = g10.png	 **k** = g11.png	 **l** = g12.png
 **m** = g13.png	 **n** = g14.png	 **o** = g15.png	 **p** = g16.png	 **q** = g17.png	 **r** = g18.png

One line per character: `key = picture.png`. The 18 samples are given to a–r **only as a demo**, so re-assign them. Add more lines for the remaining letters and digits. Anything not listed stays normal text.

```
s = my_s.png          # add your own
7 = my_seven.png      # digits work the same way
U+0023 = hash.png     # U+ codes for special keys such as # or =
```

The same file also works in the Windows editor. Each program quietly ignores the other’s settings.

7. Customize the look

What	Where	Change
Letter size	`glyphs.txt`	`@android_text_sp=28`
Width of each glyph box	`glyphs.txt`	`@android_cell_width=0.9` (fraction of the text size; bigger = wider spacing)
How big the picture is inside its box	`glyphs.txt`	`@android_glyph_scale=0.9` (0.1–1.0)
Capital letters	`glyphs.txt`	`@case_insensitive=1` shares the picture; `0` + lines like `A = a_cap.png` gives capitals their own
Page and text colour	`Config.kt`	`BACKGROUND_COLOR`, `TEXT_COLOR` (e.g. `Color.parseColor("#102030")`)
Glyph (ink) colour	tool	`python tools/glyph_tool.py normalize raw glyphs --color 1A1AFF`
Margins, title	`Config.kt`	`PADDING_DP`, `APP_TITLE`
Font for normal characters	`MainActivity.kt`	`edit.typeface = Typeface.MONOSPACE` (try `Typeface.SERIF`)

Changes in `glyphs.txt` inside `assets` and in `Config.kt` need a rebuild. Changes to a `glyphs.txt` in the app's own folder (place 2) need only a restart of the app.

8. Prepare new pictures

```
python tools/glyph_tool.py normalize my_raw_pictures app/src/main/assets/glyphs --size 96
```

It trims the empty border, centres each glyph in a square, makes the white background transparent, and keeps small PNG files. Name each raw picture as you want it called in `glyphs.txt` (`s.png`, `seven.png`). Needs Python and `pip install pillow`.

9. Using the app

Menu	What it does
Save	Saves to the open file; if it is a new note, asks where (Save as).
Glyphs / plain	Switches between your glyphs and ordinary letters (the title shows "[plain]").
New / Open… / Save as…	Standard file actions through Android's own file picker (works with Drive, Downloads, SD card).
About / glyph folder	Shows the glyph source and how many pictures were loaded.

- An open file is saved automatically when you leave the app, and unsaved text is kept as a draft and restored next time.
- Files are saved as normal UTF-8 text. Other apps (WhatsApp, Notes) will show the real letters; only this app shows your glyphs.

10. If something goes wrong

Problem	Fix
Gradle sync fails	Check internet. If Android Studio offers to update the Gradle or Android plugin, accept. Send me the message if it still fails.
“SDK location not found”	Open **File → Project Structure → SDK Location**, or install the SDK from the SDK Manager (platform 34).
A dialog lists pictures that could not be loaded	The file name in `glyphs.txt` does not match a PNG in the folder. Check spelling and capital letters (Android is case sensitive: `G01.png` ≠ `g01.png`).
Letters stay normal	The key is not in `glyphs.txt`, or the menu is in plain mode.
Phone keyboard behaves oddly	Suggestions are already switched off. Try another keyboard app, or tell me which one and what happens.
Glyphs overlap or look too small	Raise `@android_cell_width` and/or `@android_glyph_scale`.

11. Limits and next steps

- Letters are shown in a fixed box each; very long lines wrap like normal text.
- No undo/redo yet (an easy addition).
- Next ideas: a glyph keyboard strip, share-as-image of your glyph text, a signed release APK for the Play Store.

Your glyph pictures stay on your phone. The app has no internet permission.